Open Notebook Project — Modernization & Efficiency Prompts Master Tracker

Comprehensive 45-Packet Implementation, Test Verification, and Audit Master Matrix | Format: Landscape LibreOffice / MS Word

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#01 (L0.1)	**Pin Upstream Provenance**	Lock in the exact version of the upstream Open Notebook source code so we always know what version we are starting from.	Prevents upstream drift and untracked changes from breaking downstream native builds, guaranteeing 100% reproducible releases.	• Inspect the current upstream checkout discovery and runtime-preparation path without changing behavior. • Choose a small lock/manifest representation for the upstream revision and relevant toolchain provenance. • Make runtime preparation fail closed when the configured upstream revision cannot be resolved or does not match. • Record the exact revision in the generated build/runtime manifest. • Add focused tests for matching, missing, and mismatched revisions. • Document the new provenance artifact in the packet handoff.	• Run the existing wrapper test suite relevant to runtime preparation. • Run new provenance/lock tests. • Run git diff --check and inspect the targeted diff.	**PM / LLM Verification:** Execute packet #01 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ _____ Reproduced: [ ] Y [ ] N Action / Fix: _____ _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#02 (L0.2)	**Capture Baseline Metrics**	Record initial memory, start-up times, installer sizes, and resource consumption before changing any code.	Establishes a quantitative ground-truth baseline to objectively prove memory and speed improvements across modernization tiers.	• Build the application from the pinned source using the current architecture. • Record installer size and installed size. • Record process count, idle working set/RSS, idle CPU, cold-start-to-window, cold-start-to-API-ready, and shutdown time. • Run representative document ingestion/search/RAG workloads and capture peak memory and latency. • Capture long-session/orphan-process observations where practical. • Store raw results under artifacts/modernization/benchmarks/ in a machine-readable form.	• Confirm the baseline can be rebuilt from the recorded provenance. • Confirm benchmark artifacts are parseable and include units/environment metadata.	**PM / LLM Verification:** Execute packet #02 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ _____ Reproduced: [ ] Y [ ] N Action / Fix: _____ _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#03 (L0.3a)	**Add TypeScript Tooling**	Set up strict typing and boundary definitions for the application's wrapper scripts.	Eliminates dynamic JavaScript typing bugs and establishes reliable type contracts across process and service boundaries.	• Add TypeScript configuration/tooling compatible with the current wrapper build and tests.
				• Define core interfaces/types for services, runtime manifests, runtime resolution, problem reports, process handles, shutdown results, and related boundary data.
				• Keep existing JavaScript runtime paths operational during the transition.
				• Add typechecking to the validation path without forcing unrelated file conversion.
				• Create or update fixtures only where needed to express the contracts.
					• Run TypeScript typecheck.
					• Run existing tests to prove no behavior change.
					• Verify packaging/build scripts still execute as before.
						PM / LLM Verification:
						Execute packet #03 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#04 (L0.3b)	**Port Pure Wrappers**	Convert standalone utility and helper scripts from loose JavaScript into strict TypeScript.	Ensures pure helper utilities are fully type-safe and verified by the compiler without affecting runtime side-effects.	• Identify the smallest set of pure modules such as runtime manifest parsing, runtime resolution, URL/window policy, encryption-key/config helpers, or problem-report formatting.
				• Port only the selected modules in this packet.
				• Preserve public exports or provide a compatibility shim so callers do not change simultaneously.
				• Convert existing fixtures into language-neutral JSON where useful.
				• Add differential tests comparing old and new behavior until the old implementation can be retired safely.
					• Typecheck.
					• Run targeted module tests.
					• Run the existing wrapper regression suite.
					• Run differential fixture comparisons where created.
						PM / LLM Verification:
						Execute packet #04 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date:`
						`Notes:`
							Failure Audit Log:
							`Issue ID:`
							`Root Cause:`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix:`

#05 (L0.3c)	**Port Process Managers**	Convert background process-handling and service-spawning scripts into type-safe TypeScript.	Prevents catastrophic process-leak errors, invalid CLI argument handling, and daemon-spawning crashes across operating systems.	• Inventory the process-management modules and current startup/shutdown contract.
				• Port one coherent process-management boundary at a time.
				• Preserve service ordering, readiness checks, logging, shutdown semantics, and error propagation.
				• Keep process termination rules explicit and testable.
				• Export language-neutral service/supervision fixtures for later Rust/Go/C# candidates.
					• Typecheck.
					• Run supervisor/service-table tests.
					• Run shutdown/orphan-process tests.
					• Run a smoke launch with the current Electron host.
						PM / LLM Verification:
						Execute packet #05 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date:`
						`Notes:`
							Failure Audit Log:
							`Issue ID:`
							`Root Cause:`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix:`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#06 (L0.4)	**Build Replacement Harness**	Create a unified testing framework that tests host behavior regardless of whether it's written in Electron, Tauri, or C#.	Allows swapping out host technologies cleanly while verifying that every required IPC and lifecycle contract still passes identically.	• Define canonical JSON fixtures for pure behavior.
				• Implement differential-test adapters for the current reference implementation.
				• Add property-based cases for path/config/manifest/policy behavior where appropriate.
				• Create process-supervision scenarios for startup, failure, shutdown, crash, stale-port, and restart behavior.
				• Define a machine-readable compatibility result format.
				• Document how a new native-host candidate plugs into the harness.
					• Run the harness against the current implementation and require a clean reference result.
					• Verify intentionally corrupted fixtures fail.
						PM / LLM Verification:
						Execute packet #06 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date:`
						`_____`
						`Notes:`
						`_____`
						`_____`
						`_____`
							Failure Audit Log:
							`Issue ID:`
							`_____`
							`Root Cause:`
							`_____`
							`_____`
							`_____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix:`
							`_____`
							`_____`
							`_____`

#07 (L0.5)	**Create Performance Harness**	Build automated benchmarking tools to measure CPU, memory, and launch latency continuously.	Catches performance regressions immediately in CI instead of finding out after release that an installer is bloated or sluggish.	• Implement repeatable startup timing measurement.
				• Implement process-tree CPU/memory sampling for idle and workloads.
				• Capture installer/installed-size data.
				• Add representative ingestion/search/RAG workload timing hooks.
				• Record shutdown/orphan-process outcomes.
				• Emit normalized benchmark artifacts with environment metadata.
				• Make candidate-vs-baseline comparison easy to automate.
					• Run the harness twice against the unchanged baseline and quantify measurement noise.
					• Verify missing metrics fail loudly rather than becoming zero/null silently.
						PM / LLM Verification:
						Execute packet #07 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date:`
						`_____`
						`Notes:`
						`_____`
						`_____`
						`_____`
							Failure Audit Log:
							`Issue ID:`
							`_____`
							`Root Cause:`
							`_____`
							`_____`
							`_____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix:`
							`_____`
							`_____`
							`_____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#08 (L0.6)	**Port-Conflict Safe Recovery**	Detect if background server ports (like 8000 or 5055) are already in use, and give users an interactive way to fix it.	Stops confusing launch-time silent failures when leftover instances or other apps occupy necessary local service ports.	• Define language-neutral conflict/process/ownership result structures first.
				• Implement the interim JS/TypeScript port-conflict module using platform-appropriate process/port inspection.
				• Determine executable path and command context strongly enough to distinguish app-owned from unrelated processes.
				• Require explicit user confirmation before any termination.
				• Revalidate PID/ownership immediately before killing to reduce PID-reuse/TOCTOU risk.
				• Integrate bounded retry into service startup and Electron UI.
				• Add diagnostic reporting that redacts sensitive paths/data where required.
				• Add real-listener, mock-dialog, unrelated-process, ambiguity, retry-limit, and restart tests.
					• Run targeted port-conflict tests.
					• Run supervisor/chaos tests.
					• Verify an unrelated process on a required port is never auto-killed.
					• Verify app-owned orphan cleanup permits successful retry.
						PM / LLM Verification:
						Execute packet #08 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#09 (L1.1)	**Inventory Package Dependencies**	Audit all Python and Node.js packages to find huge libraries and redundant dependencies.	Identifies the heaviest disk and memory culprits in the installer bundle to prioritize surgical slimming.	• Generate recursive size reports for packaged runtime directories. • Generate Python package size and dependency reports. • Generate frontend/Node dependency size reports. • Record representative Python imports during core workloads. • Classify packaged components as required, optional, transitive-only, duplicate, or candidate for removal. • Produce an allowlist-oriented packaging inventory.	• Re-run reports from a clean build and confirm stable totals within expected noise. • Cross-check totals against actual installer/installed size.	**PM / LLM Verification:** Execute packet #09 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ Reproduced: [ ] Y [ ] N Action / Fix: _____
#10 (L1.2)	**Validate No-Docling Runtime**	Ensure the application can boot and run its core notebook features without having to load the heavy Docling document parser.	Docling accounts for over 1.2 GB of disk and hundreds of MBs of RAM; separating it makes default app operation lightweight and instant.	• Enumerate the exact workflows that currently invoke Docling. • Measure Docling and transitive dependency footprint. • Add an explicit feature flag/configuration for Docling capability. • Produce a core runtime variant that excludes Docling dependencies. • Run normal non-Docling ingestion/search/RAG workflows. • Record behavior when a Docling-required workflow is requested but the component is unavailable.	• Run standard-workflow regression tests against both full and core-only variants. • Compare package size and idle/startup metrics.	**PM / LLM Verification:** Execute packet #10 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ Reproduced: [ ] Y [ ] N Action / Fix: _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#11 (L1.3)	**Demand-Load Docling Parser**	Load the heavy document parsing library only when a user actually uploads an advanced document that requires it.	Cuts idle RAM usage drastically by keeping large ML weights out of memory during general notebook editing and browsing.	• Choose one prototype: optional installer component, separate environment/process, or first-use acquisition consistent with offline/privacy requirements. • Keep the core runtime independent of Docling imports at idle. • Define lifecycle/error behavior for missing optional capability. • If process isolation is used, ensure the heavy process exits and memory is reclaimed after work. • Create a golden extraction corpus covering Docling-relevant cases. • Compare output quality and errors against the current full runtime.	• Run golden extraction comparisons. • Measure core install size and idle memory. • Measure peak extraction memory and post-job reclamation.	**PM / LLM Verification:** Execute packet #11 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ Reproduced: [ ] Y [ ] N Action / Fix: _____
#12 (L1.4)	**Profile Python Memory**	Analyze duplicated module imports and memory overlap between the FastAPI server and the background worker.	Removes duplicate in-memory libraries between Python processes to free tens to hundreds of megabytes of baseline RAM.	• Capture import-time profiles for API and worker. • Capture per-process idle memory after warm startup. • Identify overlapping heavy modules and optional imports. • Identify modules that can be lazy-loaded without behavior changes. • Prototype only clearly safe lazy imports in a separate small patch if evidence supports it. • Produce a ranked table of retained-memory contributors.	• Run API/worker startup tests before and after any lazy-loading patch. • Compare idle memory and first-use latency.	**PM / LLM Verification:** Execute packet #12 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ Reproduced: [ ] Y [ ] N Action / Fix: _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#13 (L1.5)	**Benchmark Worker Concurrency**	Test different background worker configurations to find the best balance of speed versus memory usage on a laptop.	Prevents multi-worker thread pools from saturating laptop CPU fans and exhausting user system memory.	• Identify current max-task/concurrency settings and queue semantics.
				• Define representative light, mixed, and heavy workloads.
				• Benchmark a small bounded matrix of concurrency settings.
				• Measure throughput, latency, peak memory, CPU, failure/retry behavior, and UI/API responsiveness.
				• Recommend a default based on evidence rather than maximum throughput alone.
					• Run each candidate enough times to distinguish meaningful differences from noise.
					• Verify job correctness/retry behavior remains unchanged.
						PM / LLM Verification: Execute packet #13 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

#14 (L1.6)	**On-Demand Worker Lifecycle**	Keep the background task worker asleep and only spin it up when background tasks are queued.	Drastically reduces battery drain and idle footprint on laptops when users are just reading notes.	• Define the trigger that starts the worker and the conditions that keep it alive.
				• Define idle timeout and shutdown behavior.
				• Preserve queue lease/retry/status semantics.
				• Prototype supervisor-controlled worker start/stop without rewriting workflow handlers.
				• Test multiple sequential jobs, concurrent enqueue, worker crash, host restart, and idle exit.
				• Measure cold-job latency, idle memory savings, and peak memory.
					• Run queue/job correctness tests.
					• Run crash/restart scenarios.
					• Compare idle and workload metrics with baseline.
						PM / LLM Verification: Execute packet #14 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#15 (L2.1)	**Freeze Native Contract**	Document and freeze all the exact API and IPC calls that any native desktop window must support.	Creates an unambiguous specification so any native framework candidate can be evaluated without shifting goals.	• Specify runtime discovery and verification inputs/outputs. • Specify service launch ordering, environment construction, logging, readiness, shutdown, and problem-report semantics. • Specify navigation/external-link policy and loopback network restrictions. • Specify data/config paths and encryption-key handling. • Include the safe port-conflict recovery contract. • Specify required benchmark and compatibility-harness integration. • Version the host configuration schema.	• Validate the contract against the current implementation fixtures. • Require each ambiguous behavior to be resolved by test or explicit decision record.	**PM / LLM Verification:** Execute packet #15 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ Reproduced: [ ] Y [ ] N Action / Fix: _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#16 (L2.2)	**Build Rust-Tauri Candidate**	Build a prototype desktop app shell using Rust and Tauri to evaluate its speed and size.	Tests the ultra-lightweight webview approach to see if it meets stability and integration requirements.	• Scaffold the smallest viable Tauri host.
				• Reproduce window/navigation security policy.
				• Implement runtime-path resolution and verification.
				• Launch the existing services without moving business logic.
				• Implement readiness checks and shared port-conflict contract.
				• Add Windows Job Object containment or an audited equivalent.
				• Add structured logging/problem-report output.
				• Connect the candidate to the shared compatibility/performance harness.
					• Run host contract suite.
					• Run crash/shutdown/stale-port scenarios.
					• Benchmark startup/idle/process count against Electron.
						PM / LLM Verification: Execute packet #16 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`_____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`
							`_____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#17 (L2.3)	**Build Go-Wails Candidate**	Build a prototype desktop app shell using Go and Wails to evaluate its speed and size.	Evaluates Go's concurrency and single-binary deployment benefits as an alternative host candidate.	• Scaffold a minimal Wails host with the existing React concept. • Implement navigation/window security policy. • Implement runtime discovery/verification and process supervision. • Implement the shared port-conflict recovery contract. • Add Windows Job Object containment. • Keep any static/proxy experiments behind flags; do not cross into Section 19. • Run Go race detection and shared compatibility/chaos harnesses.	• Run go test -race ./... where applicable. • Run host contract and chaos suites. • Benchmark against Electron and the Rust candidate using identical scenarios.	**PM / LLM Verification:** Execute packet #17 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____ _____ _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ _____ _____ Reproduced: [ ] Y [ ] N Action / Fix: _____ _____ _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#18
(L2.4)
	Build WinUI-WebView2 Candidate	Build a prototype desktop app shell using C# and WinUI/WebView2 for native Windows optimization.	Leverages native Windows APIs and the built-in system Edge WebView2 to eliminate shipping third-party browser binaries.	• Scaffold a WinUI desktop host with WebView2.
				• Load the existing frontend/server path without changing Node architecture.
				• Implement navigation/window policy.
				• Implement runtime discovery/verification and child-process supervision.
				• Add Windows Job Object containment.
				• Keep static/proxy experiments behind flags only; do not cross into Section 19.
				• Compare conventional/self-contained/AOT deployment only where the prototype supports it safely.
					• Run host contract and chaos suites.
					• Benchmark startup, idle memory, process count, and package impact.
						PM / LLM Verification:
						Execute packet #18 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#19 (L2.5)	**Score Host Candidates**	Evaluate Rust, Go, and C# prototypes with objective metrics and pick the winning production desktop host.	Ensures architectural decisions are backed by data (startup time, installer size, maintainability) rather than speculation.	• Normalize candidate scorecards against the same baseline and environment.
				• Compare contract-suite pass/fail results.
				• Compare crash/restart/orphan-process behavior.
				• Compare startup, idle memory/CPU, process count, package size, and build complexity.
				• Record Windows integration and long-term maintenance tradeoffs.
				• Write an ADR selecting one candidate or explicitly requesting another prototype if evidence is inconclusive.
					• Re-run any metric whose measurement conditions differ between candidates.
					• Require zero unresolved correctness/security failures in the selected candidate.
						PM / LLM Verification: Execute packet #19 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

#20 (L3.1)	**Inventory Next.js Responsibilities**	Catalog everything the Next.js web server does to see what can be turned into simple static files.	Prepares the frontend to run without a background Node.js server, which is essential to cutting out Node.js entirely.	• Classify routes/components as static build concern, client routing concern, API proxy concern, SSE/streaming concern, upload/body concern, server-only framework dependency, or not needed on desktop.
				• Identify every desktop-specific rewrite/proxy responsibility.
				• Separate UI dependencies from server-runtime dependencies.
				• Build a route-compatibility map.
				• Identify blockers to a static/client build.
				• Do not modify production routing in this packet.
					• Cross-check route inventory against source tree and existing tests.
					• Have an independent/fresh-context review look for missed server-only behavior.
						PM / LLM Verification: Execute packet #20 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#21 (L3.2)	**Freeze Proxy Fixtures**	Create automated verification recordings of all API proxies, file uploads, and live streaming data.	Guarantees that replacing Next.js server proxies with native host proxies won't break live AI chat streaming or file uploads.	• Record representative proxied API requests/responses, status codes, and relevant headers. • Capture upload/body behavior and error cases. • Capture SSE framing, incremental timing, connection close, and cancellation behavior. • Create stable synthetic fixtures where live model/service behavior would be nondeterministic. • Add executable compatibility tests that can target current Next and future native proxy implementations.	• Run fixtures against the current Next implementation and require a clean reference result. • Verify an intentionally buffered/altered SSE implementation would fail the test.	**PM / LLM Verification:** Execute packet #21 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ Reproduced: [ ] Y [ ] N Action / Fix: _____
#22 (L3.3)	**Build Static Frontend**	Convert the web interface into pure HTML/CSS/JS that can be loaded straight from the native host without Node.	Proves the app can function completely as a client-side bundle served by native code.	• Create a parallel static/client build target rather than replacing production build immediately. • Port routing only as needed for the proof of concept. • Replace server-only environment access with an explicit client-safe configuration path. • Point API calls at a configurable host/proxy endpoint without implementing Section 19. • Implement SPA fallback behavior in the proof-of-concept environment. • Run UI, API, upload, and streaming regressions. • Record blockers instead of broad refactoring.	• Build both current Next and static/client targets. • Run frontend regression/smoke tests. • Run proxy/SSE fixtures through the existing server path.	**PM / LLM Verification:** Execute packet #22 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ Reproduced: [ ] Y [ ] N Action / Fix: _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#23 (L3.4)	**Prepare Native Handoff**	Bundle static web assets and proxy definitions for clean integration into the native application host.	Establishes a rock-solid handoff point between web frontend developers and native host systems.	• Package the Next server dependency inventory.
				• Package static/client build artifacts/instructions and unresolved blockers.
				• Package API, upload, and SSE compatibility fixtures.
				• Include selected host contract and security constraints.
				• Include current benchmark baseline and rollback build instructions.
				• List exact commits and repository state expected at the handoff boundary.
				• Update CURRENT.md to state that the next approved work is external gate X19.
					• Verify every referenced artifact/path exists.
					• Run the frozen proxy/SSE suite one final time against the current implementation.
						PM / LLM Verification:
						Execute packet #23 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____ _____ _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____ _____ _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____ _____ _____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#24 (X19)	**Host-Managed Frontend Serving**	Have the native desktop app directly serve the user interface and route API calls to the Python backend.	Removes the need for a web server process running in the background, slashing process count and latency.	• Implement static asset serving with correct path, MIME, cache, and traversal protections.
				• Implement a reverse proxy restricted to the intended loopback backend target.
				• Preserve methods, bodies/uploads, required headers, status codes, cancellation, and error behavior.
				• Implement SSE as true incremental pass-through with no buffering regression.
				• Apply deliberate request limits/timeouts without breaking established contracts.
				• Run differential API/proxy/static/streaming tests against the current implementation.
				• Keep a rollback build available.
					• Static-serving security tests.
					• Proxy/upload compatibility suite.
					• SSE framing/timing/cancellation suite.
					• Benchmark host overhead and startup impact.
						PM / LLM Verification:
						Execute packet #24 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date:`
						`Notes:`
							Failure Audit Log:
							`Issue ID:`
							`Root Cause:`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix:`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#25 (X20)	**Remove Bundled Node.js**	Completely remove Node.js from the application distribution bundle.	Eliminates ~50–80 MB of packaged binary bloat and removes an entire runtime engine and potential security attack surface.	• Identify every remaining runtime dependency on Node/Next standalone server.
				• Remove Node from runtime preparation, manifest, service table, packaging, and installer inputs.
				• Ensure static frontend assets are served entirely by the selected native host.
				• Remove dead Node-specific startup/recovery paths only after tests prove they are unused.
				• Run clean-machine validation that fails if a hidden system Node dependency exists.
				• Update architecture state and package-size benchmarks.
				• Preserve rollback to the pre-X20 build until acceptance.
					• Clean machine/VM launch with no Node installed.
					• Full API/upload/SSE/UI regression suite.
					• Installer/package inventory confirms Node absence.
					• Benchmark size/startup/idle deltas.
						PM / LLM Verification: Execute packet #25 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#26 (L4.1)	**Productionize Native Host**	Turn the selected native desktop prototype into a rock-solid, production-grade application shell.	Integrates menus, window state persistence, tray icons, and robust error handling for real-world user adoption.	• Integrate production runtime discovery/verification.
				• Integrate structured logs and problem reports.
				• Integrate safe port-conflict UX and ownership checks.
				• Preserve navigation/external-link policy and data/config paths.
				• Preserve current frontend path appropriate to repository state.
				• Add a clear build/runtime fallback feature flag or variant.
				• Run the full host contract suite.
					• Host contract suite.
					• UI startup/error flows.
					• Port-conflict scenarios.
					• Benchmark against selected prototype and current production host.
						PM / LLM Verification:
						Execute packet #26 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

#27 (L4.2)	**OS-Owned Process Containment**	Use operating system job objects to ensure all Python backend processes close instantly when the app is closed.	Guarantees zero orphaned Python processes or memory leaks lingering in Task Manager after quitting.	• Implement Windows Job Object ownership or the selected language's audited wrapper.
				• Attach all app-owned child processes at launch.
				• Set kill-on-host-close semantics appropriate to the design.
				• Preserve graceful shutdown first, with containment as the final guarantee.
				• Remove or demote redundant ad hoc tree-kill logic only after parity tests pass.
				• Document any child process that intentionally cannot join the job.
					• Graceful close.
					• Host crash/force kill.
					• Child crash.
					• Immediate restart after crash.
					• Verify no app-owned required-port listener survives.
						PM / LLM Verification:
						Execute packet #27 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#28 (L4.3)	**Improve Service Readiness**	Make startup wait intelligently for Python servers to report healthy before showing the window.	Prevents unsightly blank pages, connection-refused errors, and spinner freezes on cold start.	• Document current readiness definition for each service. • Separate startup readiness from ongoing health. • Add richer checks one service at a time, keeping port-open fallback until proven. • Bound retry/backoff behavior and preserve useful failure context. • Ensure health checks do not create material idle CPU/network noise.	• Startup success/failure tests per service. • Crash-after-ready tests. • Idle CPU/network comparison. • Problem-report verification.	**PM / LLM Verification:** Execute packet #28 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ _____ Reproduced: [ ] Y [ ] N Action / Fix: _____ _____
#29 (L4.4)	**Retire Electron Shell**	Remove Electron completely while keeping an emergency fallback build script just in case.	Finalizes the removal of heavy Chromium/Electron overhead while maintaining release safety.	• Reproduce window sizing/lifecycle and startup/error UX. • Reproduce navigation and external-link policy. • Ensure service lifecycle and problem reporting are equivalent. • Remove Electron runtime/package inputs from the native production variant. • Build installer/package with the native host. • Run clean-machine and upgrade-path tests. • Keep Electron rollback variant for the defined canary period.	• Full host contract. • UI smoke/regression. • Clean VM install/start/shutdown. • Package-size/startup/idle benchmark. • Security navigation tests.	**PM / LLM Verification:** Execute packet #29 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ _____ Reproduced: [ ] Y [ ] N Action / Fix: _____ _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#30 (L4.5)	**Native Chaos Security**	Stress-test the desktop app by killing backend processes unexpectedly and auditing local security.	Ensures the application recovers gracefully from crashes and is secure against local privilege escalation or injection.	• Run crash/force-kill/restart scenarios.
				• Run stale-port scenarios for app-owned, unrelated, and ambiguous processes.
				• Test path edge cases and runtime manifest tampering.
				• Verify loopback-only exposure and navigation isolation.
				• Review logs/problem reports for secret/path leakage.
				• Record every failure as a reproducible test or tracked exception.
					• Shared chaos suite.
					• Security tests.
					• Manual review of problem-report/log samples.
						PM / LLM Verification: Execute packet #30 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

#31 (L5.1)	**Modularize Runtime Packaging**	Split large components (like AI models or document parsers) into optional downloadable add-ons.	Enables a small, fast initial installer while letting users download heavy components only if needed.	• Define packaging units: core host/frontend, SurrealDB, Python core backend, optional extraction/Docling, and optional extras where justified.
				• Give each component explicit inputs, version/provenance, and output paths.
				• Generate a machine-readable package manifest.
				• Prevent accidental inclusion of unallowlisted runtime content.
				• Preserve current full/offline build as a comparison variant during transition.
					• Build full and modular variants.
					• Compare manifests and functional smoke tests.
					• Verify component omission fails clearly when required.
						PM / LLM Verification: Execute packet #31 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#32 (L5.2)	**Experiment Installer Variants**	Build and compare different installer packages (MSI, MSIX, NSIS, portable zip) for installation speed and cleanliness.	Finds the smoothest, most reliable installation experience for end users across various enterprise and personal environments.	• Define a bounded set of installer variants using the modular package graph. • Include optional Docling/extraction handling where supported. • Define behavior for missing shared runtimes such as WebView2 if relevant. • Automate install, repair/upgrade, and uninstall checks. • Measure installer size, installed size, install time, and failure/recovery behavior. • Select a preferred variant or record why current packaging remains best.	• Automated installer smoke tests. • Upgrade/uninstall/reinstall tests. • Offline behavior tests where required. • Size/time benchmark.	**PM / LLM Verification:** Execute packet #32 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ _____ Reproduced: [ ] Y [ ] N Action / Fix: _____ _____
#33 (L5.3)	**Validate Clean-Machine Installs**	Test installation on clean Windows machines that do not have Python, Node, or developer tools pre-installed.	Ensures real users never encounter 'missing DLL' or 'command not found' errors on fresh consumer PCs.	• Create or use clean VM snapshots for supported Windows versions. • Test fresh install, first launch, normal workload, shutdown, uninstall, and reinstall. • Test upgrade from the last supported release path. • Test path/username/space/non-default install edge cases. • Test missing shared prerequisites and recovery messaging. • Confirm no unexpected Node/Electron dependency remains when those removals are complete.	• Record machine/environment IDs and exact installer hashes. • Require zero orphan services/processes after uninstall/shutdown.	**PM / LLM Verification:** Execute packet #33 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ _____ Reproduced: [ ] Y [ ] N Action / Fix: _____ _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#34 (L6)	**Re-Profile Simplified App**	Measure memory, disk space, and speed of the modernized application against the original baseline.	Demonstrates clear, measurable ROI (e.g. 70% less RAM, 5x faster boot) achieved from the modernization effort.	• Measure process tree, startup, idle memory/CPU, installer/installed size, and shutdown. • Measure ingestion/search/RAG and extraction workloads. • Measure worker memory retention and optional extraction spikes. • Compare all metrics to untouched baseline and intermediate milestones. • Rank current bottlenecks by measured impact. • For each bottleneck, classify it as CPU, allocation/copying, I/O, network, model inference, database, or packaging overhead. • Do not implement optimizations in this packet.	• Repeat benchmark runs enough to establish stable deltas. • Verify benchmark environment matches or normalize differences explicitly.	**PM / LLM Verification:** Execute packet #34 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ Reproduced: [ ] Y [ ] N Action / Fix: _____
#35 (L7.1)	**Cython CPU Optimization**	Compile the single most CPU-intensive Python computation into high-speed C code using Cython.	Boosts document processing and calculation throughput without rewriting the whole application in C.	• Select exactly one hotspot supported by L6 profiling. • Freeze its public Python API and golden inputs/outputs. • Implement a Cython version with the smallest practical surface. • Keep a Python fallback during evaluation. • Benchmark micro and end-to-end workload impact. • Audit packaging/toolchain impact before promotion.	• Golden behavior tests. • Fallback parity tests. • Microbenchmark plus end-to-end benchmark. • Build/package test.	**PM / LLM Verification:** Execute packet #35 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ Reproduced: [ ] Y [ ] N Action / Fix: _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#36 (L7.2)	**Rust Extension Acceleration**	Speed up a proven bottleneck in Python by replacing it with a lightning-fast Rust extension library.	Delivers native C/C++ speeds with memory safety for heavy vector or text processing operations.	• Select one leaf hotspot and freeze its Python API.
				• Create golden and edge-case parity tests.
				• Implement the smallest PyO3/maturin-style native surface appropriate to the repo.
				• Keep a Python fallback until accepted.
				• Benchmark boundary overhead, allocations, and end-to-end workload impact.
				• Audit packaging and cross-version compatibility.
					• Parity/property tests.
					• Native build test.
					• Micro and end-to-end benchmarks.
					• Fallback test.
						PM / LLM Verification:
						Execute packet #36 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date:`
						`Notes:`
							Failure Audit Log:
							`Issue ID:`
							`Root Cause:`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix:`

#37 (L7.3)	**Integrate Native Libraries**	Link proven, high-performance C/C++ libraries only where specialized binary acceleration is needed.	Reuses battle-tested high-speed native libraries rather than reinventing complex algorithms in Python.	• Document why C/C++ is required instead of Rust/Cython/existing bindings.
				• Freeze the narrow input/output contract.
				• Prefer wrapping an established library over authoring new unsafe code.
				• Add memory-safety tooling appropriate to the build.
				• Add malformed-input and lifecycle tests.
				• Measure whether the integration produces a meaningful application-level win.
					• Golden/parity tests.
					• Sanitizer or equivalent memory-safety checks where supported.
					• Malformed input tests.
					• End-to-end benchmark.
						PM / LLM Verification:
						Execute packet #37 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date:`
						`Notes:`
							Failure Audit Log:
							`Issue ID:`
							`Root Cause:`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix:`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#38 (L7.4)	**Evaluate Helper Binaries**	Determine whether small Go or C# standalone utilities make sense for isolated background tasks.	Avoids architectural complexity by ensuring separate helpers are only built if they genuinely simplify architecture.	• Define the helper's single responsibility and IPC/CLI contract.
				• Justify why it should be a separate process/binary.
				• Implement in the language that best matches the OS/platform boundary.
				• Add timeout/error/version compatibility behavior.
				• Measure startup/process/package overhead to ensure the helper does not erase the intended gain.
					• Contract tests.
					• Failure/timeout tests.
					• Package/process-overhead measurement.
						PM / LLM Verification: Execute packet #38 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

#39 (L7.5)	**Study Worker Migration**	Conduct a feasibility study on rewriting background workers in native code without prematurely coding it.	Prevents risky, expensive rewrites by analyzing real trade-offs between Python simplicity and native speed first.	• Map every worker command handler to downstream Python modules/workflows.
				• Identify queue/lease/retry/status responsibilities separately from business workflows.
				• Quantify what a native coordinator could replace while leaving workflows in Python.
				• Estimate IPC/process-start costs for short-lived Python job executors.
				• Estimate upstream synchronization burden of deeper rewrites.
				• Produce options: keep current, on-demand Python, native coordinator + Python executor, or deeper rewrite.
				• Do not implement the worker rewrite in this packet.
					• Cross-check dependency map against source imports/call paths.
					• Review the study against measured L6 bottlenecks.
						PM / LLM Verification: Execute packet #39 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#40 (L7.6)	**Study FastAPI Native**	Research whether the FastAPI backend should eventually become a native Go or Rust server.	Provides an informed long-term roadmap for future performance gains without derailing current stability.	• Group API routes by downstream dependency graph.
				• For each route group, determine whether replacing HTTP handling would remove any Python process/dependency.
				• Identify new IPC/serialization/streaming complexity a native front would introduce.
				• Estimate upstream-sync and test burden.
				• Estimate realistic resource wins from measured data.
				• Define parity tests required for any route group worth prototyping.
				• Do not implement a native backend in this packet.
					• Validate route/dependency mapping against code.
					• Reject proposals lacking a measurable end-state benefit.
						PM / LLM Verification: Execute packet #40 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

#41 (L8.1)	**Audit Network Isolation**	Audit localhost network sockets and browser navigation to prevent malicious websites from accessing local data.	Secures local user data and prevents cross-site request attacks against the local notebook server.	• Enumerate every listening socket and bind address.
				• Verify required services bind only as intended.
				• Test proxy target pinning and path traversal/host-header edge cases.
				• Test WebView navigation allow/deny behavior and external-link handling.
				• Add fuzz/property cases for URLs/paths where practical.
				• Convert every discovered regression into an automated test.
					• Port/listener inspection.
					• Navigation tests.
					• Proxy/path security tests.
					• Fuzz/property tests where implemented.
						PM / LLM Verification: Execute packet #41 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#42 (L8.2)	**Audit Secrets Licenses**	Scan the entire project for hardcoded API keys, stray local paths, third-party licenses, and diagnostic leaks.	Protects user privacy, ensures legal open-source compliance, and prevents sensitive credentials from leaking into releases.	• Trace encryption-key and secret handling through host, environment, logs, and diagnostics. • Review problem reports for accidental secret or sensitive-path leakage. • Audit runtime/download/build dependencies and pinned versions. • Generate license inventory for redistributed components. • Run the project's chosen vulnerability scanners and triage findings. • Add redaction tests for newly identified sensitive fields.	• Secret/redaction tests. • Dependency/vulnerability scan. • License inventory consistency check.	**PM / LLM Verification:** Execute packet #42 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ _____ Reproduced: [ ] Y [ ] N Action / Fix: _____ _____
#43 (L8.3)	**Run Golden Regression**	Run standard end-to-end tests across note creation, search, RAG, and AI summaries to verify nothing broke.	Guarantees that every core user feature works with 100% precision after massive architectural refactoring.	• Run the golden document corpus across supported extraction paths. • Run retrieval/RAG regression scenarios with stable fixtures/evaluation criteria. • Run API contract tests including errors, uploads, and streaming. • Classify every difference as expected, tolerated, or release-blocking. • Update golden data only with an explicit rationale and review record.	• Golden extraction suite. • RAG regression suite. • API contract suite.	**PM / LLM Verification:** Execute packet #43 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ _____ Reproduced: [ ] Y [ ] N Action / Fix: _____ _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)
#44 (L8.4)	**Finalize CI Release Gates**	Set up strict automated GitHub/CI test gates, rollback mechanisms, and canary release procedures.	Ensures broken builds never reach users and enables rapid one-click rollbacks if an unexpected issue arises.	• Finalize fast, integration, stress/chaos, RAG, packaging, and performance CI lanes. • Define which lanes block pull requests versus releases. • Encode release acceptance gates for correctness, security, process cleanup, packaging, and performance. • Verify replacement implementations remain switchable until promotion criteria are met. • Document canary release and rollback triggers. • Test rollback procedures rather than merely documenting them.	• Run representative CI lanes end-to-end. • Exercise at least one rollback from the release-candidate architecture.	**PM / LLM Verification:** Execute packet #44 tests. Validate output status & handoff doc. [ ] PASS [ ] FAIL Tester / Date: _____ Notes: _____	**Failure Audit Log:** Issue ID: _____ Root Cause: _____ _____ Reproduced: [ ] Y [ ] N Action / Fix: _____ _____

Prompt # & Packet	Task Title (3-4 Words)	What Was Required (Layperson Terms)	Why It Was Important	Coding Steps Needed	Tests & Audit Verification	Final Test Run (PM / LLM Instructions)	Bugs / Issues (If Test Fails)

#45 (L8.5)	**Refresh Operations Docs**	Update all architecture diagrams, deployment manuals, benchmark scorecards, and developer handbooks.	Leaves future engineers and maintainers with crystal-clear, up-to-date documentation and operating guides.	• Update architecture diagrams/process-tree documentation to match the shipped design.
				• Document build, packaging, optional components, troubleshooting, and rollback procedures.
				• Document the port-conflict recovery behavior and safety boundary.
				• Produce the final standard scorecard comparing baseline and final release candidate.
				• Record retained runtimes and why each remains.
				• Record explicitly rejected rewrites and the evidence behind those decisions.
				• Verify all packet handoffs/ADRs referenced by final docs exist.
					• Run docs link/path checks if available.
					• Re-run final benchmark scorecard on release artifacts.
					• Verify final process tree and package inventory match documentation.
						PM / LLM Verification:
						Execute packet #45 tests. Validate output status & handoff doc.
						`[ ] PASS [ ] FAIL`
						`Tester / Date: _____`
						`Notes: _____`
							Failure Audit Log:
							`Issue ID: _____`
							`Root Cause: _____`
							`Reproduced: [ ] Y [ ] N`
							`Action / Fix: _____`